

UnifAI Implementation & Configuration Guide

****Server · Windows Guard · Mac Guard · Configuration****

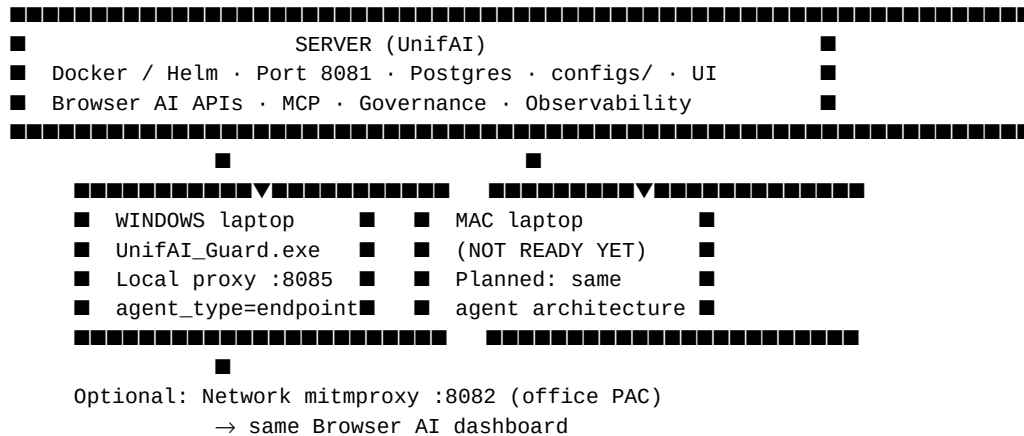
****Date:**** 10 Sep 2026

```
**Repo:** `unify - Copy` / `unifai_new`
```

```
**Docs folder:** `docs/`
```

...

1. Architecture (one picture in words)



****One dashboard:**** Workspace → ****Browser AI**** (Prompt Logs + Guard Agents + Rules + Targets).

...

2. Server implementation

2.1 What the server is

UnifAI HTTP gateway + management UI:

- AI gateway (providers, routing, MCP, governance)
- Browser AI control plane (rules, targets, agent heartbeats, PAC, logs)
- Observability plugins (Maxim, Datadog, BigQuery, Kafka, Pub/Sub, New Relic, OTEL...)

2.2 Deploy options

Method	When to use	Entry
----- ----- -----		
Docker Compose	Current company stack	<code>`docker-compose.yml`</code>
Helm	Kubernetes	<code>`deploy/helm/unifai`</code>
Local binary	Dev only	<code>`transports/unifai-http`</code>

2.3 Docker Compose (current)

****Services****

```
| Service | Role | Port |
|-----|-----|-----|
```

| `zen\_gauss\_v1` | UnifAI backend + UI APIs | `APP\_PORT` (default ****8081****) |
| `unifai\_browser\_proxy` | Shared network Guard (mitmproxy) | `PROXY\_PORT` (default ****8082****) |

****Volumes****

- `./data` → `/app/data` (runtime + config store files)
- `./configs` → `/app/configs` (pricing, ****mcp-library.json****, etc.)

****Start****

```
# From repo root, with .env filled
docker compose up -d zen_gauss_v1
# Optional network Guard proxy:
docker compose up -d unifai_browser_proxy
```

****Health checks (must return JSON, not HTML login page)****

```
GET https://<SERVER_DOMAIN>/health
GET https://<SERVER_DOMAIN>/api/browser-ai/targets
GET https://<SERVER_DOMAIN>/api/browser-ai/rules
GET https://<SERVER_DOMAIN>/api/browser-ai/proxy.pac?proxy=127.0.0.1:8085
GET https://<SERVER_DOMAIN>/api/browser-ai/pac?proxy=<corp-proxy-host>:8082
```

2.4 Server config files

File	Purpose
`.env`	Master env (domain, DB, admin, Ollama, ports)
`configs/config.json`	Framework pricing + ****mcp_library_url**** etc.
`configs/mcp-library.json`	MCP catalog (~122 servers)
`configs/pricing.json` / `model-parameters.json`	Model pricing/params
`data`	Runtime DB-backed config persistence

****MCP catalog (important)****

```
"mcp_library_url": "file:///app/configs/mcp-library.json"
```

Do ****not**** rely on `https://getunifai.ai/mcp-library` (DNS dead).

2.5 Server implementation checklist

- [] `.env` set (`SERVER\_DOMAIN`, `DB\_`, `ADMIN\_`, `OLLAMA\_URL`)
- [] Postgres reachable
- [] `docker compose up -d`
- [] `/health` → `status: ok`
- [] Login works
- [] Browser AI APIs return JSON
- [] MCP Library shows ~122 after sync
- [] Guard Agents can heartbeat to this domain

3. Windows implementation (UnifAI Guard)

3.1 Status

****Implemented and packaged.****

Version line in release notes: ****v1.6.x**** (see `apps/browser-guard/release/VERSION.txt`).

3.2 What runs on the laptop

Component	Detail
Binary	`UnifAI_Guard.exe` (PyInstaller)
Local API	`http://127.0.0.1:18085/` (agent status)
Local proxy	Typically **127.0.0.1:8085**
PAC	Applied on that PC only
`agent_type`	`endpoint`
Backend	Heartbeats + logs → `SERVER_DOMAIN` / `backend_url`

3.3 Employee install (production)

1. Fully quit old Guard (Task Manager → end all `UnifAI\_Guard.exe`).
 2. Prefer latest ****`release/UnifAI\_Guard.exe`**** matching `VERSION.txt`.
- Or company installer: ****`release/UnifAI\_Guard\_Setup.exe`****.
3. Run as Administrator once (or install via Setup).
 4. Open `http://127.0.0.1:18085/` → version + `proxy\_port` OK.
 5. UnifAI UI → Browser AI → Guard Agents → Active row shows correct version.
 6. Fully quit Chrome/Edge and reopen.
 7. Test a ****Target Website**** (prompt + optional file upload).

****Configured backend example (edit per company):****

`https://unifaiv2.dev-yp.com` or `https://unifai.dev-yp.com`

3.4 IT rebuild (Windows)

```
apps/browser-guard/  
  agent/unifai_agent.py          # agent entry  
  proxy/browser_ai_proxy.py      # mitm addon  
  UnifAI_Guard.spec              # PyInstaller  
  installer/build_installer.bat  # Inno Setup  
  release/                       # ship to employees + host Download ZIP
```

After rebuild:

1. Copy new EXE/Setup into `release/`
2. Redeploy UnifAI server so ****Download Setup ZIP**** serves new build
3. Roll out to PCs; verify Agents tab version

3.5 Hybrid (Windows laptop + office network)

See `apps/browser-guard/HYBRID\_DEPLOY.txt`.

Mode	Who	Proxy
A Laptop	Each PC EXE	`127.0.0.1:8085`
B Network	Docker `unifai_browser_proxy`	`0.0.0.0:8082`

- Office PAC/GPO → `https://<unifai>/api/browser-ai/pac?proxy=<corp-host>:8082`

- Distribute mitmproxy CA for HTTPS inspect
- ****Avoid double-MITM**** (don't force laptop PAC + corp PAC on same session)

3.6 Windows known gaps

Gap	Notes
Site coverage	Not all websites; Target list based
LLM Guard path	Often ****regex-only****; LLM classification incomplete
Chrome "all sites"	Not implemented

4. Mac implementation

4.1 Status

****Not implemented for production yet.****

Windows uses WinRT OCR + mitmproxy Windows hooks + `.exe` packaging. Mac needs a separate agent build.

4.2 Planned Mac architecture (same product behaviour)

Piece	Plan
Agent runtime	Python agent port of `unifai\_agent.py` (no WinRT-only deps)
HTTPS intercept	mitmproxy (macOS) + system/user proxy or PAC
Packaging	`.app` or signed pkg / Homebrew cask (TBD)
Backend	Same UnifAI server (`backend\_url` / `SERVER\_DOMAIN`)
Dashboard	Same Browser AI UI; `agent\_type=endpoint`
Config	Same JSON/env shape as Windows where possible

4.3 Mac implementation checklist (to build)

- [] Remove/replace Windows-only APIs (`winrt` OCR → Vision/alternative or skip OCR v1)
- [] mitmproxy CA install flow for macOS Keychain
- [] PAC / system proxy apply + remove on quit
- [] Autostart (LaunchAgent)
- [] Code signing + notarization (Gatekeeper)
- [] Local status HTTP port (parity with 18085)
- [] Heartbeat + Prompt Logs e2e against staging
- [] Employee install doc (Mac)

4.4 Temporary Mac workaround (until agent ships)

1. Use ****network Guard proxy**** (`unifai\_browser\_proxy`) on company network.
 2. Point Mac browser proxy/PAC to corp proxy.
 3. Install corp mitm CA on Mac.
 4. Mac traffic appears as ****Network / server**** agent in Browser AI (not Laptop Guard).
-

5. Configuration reference

5.1 Server `.env` (required)

Variable	Example / meaning
----- -----	
<code>`SERVER_DOMAIN`</code>	Public URL, e.g. <code>`https://unifai.dev-yp.com`</code>
<code>`APP_PORT`</code>	<code>`8081`</code>
<code>`APP_HOST`</code>	<code>`0.0.0.0`</code>
<code>`PROXY_PORT`</code>	<code>`8082`</code> (network Guard)
<code>`DB_TYPE`</code>	<code>`postgres`</code> (or <code>`mysql`</code>)
<code>`DB_HOST`</code> / <code>`DB_PORT`</code> / <code>`DB_NAME`</code> / <code>`DB_USER`</code> / <code>`DB_PASSWORD`</code>	Database
<code>`DB_SSL_MODE`</code>	<code>`disable`</code> / <code>`require`</code> / ...
<code>`ADMIN_EMAIL`</code> / <code>`ADMIN_PASSWORD`</code>	First admin
<code>`OLLAMA_URL`</code>	Guard Bot / local LLM (<code>`BROWSER_AI_OLLAMA_URL`</code> in compose)
<code>`LOG_LEVEL`</code> / <code>`LOG_STYLE`</code>	<code>`info`</code> / <code>`json`</code>
<code>`APP_DIR`</code>	<code>`/app/data`</code>

Copy from `.env.example` — *****use company secrets; do not commit real passwords.*****

5.2 Network Guard env (compose / bare metal)

Variable	Meaning
----- -----	
<code>`UNIFAI_BACKEND_URL`</code>	UnifAI API base (compose: <code>`http://zen_gauss_v1:8081`</code>)
<code>`UNIFAI_SERVER_MODE`</code>	<code>`1`</code> for network mode
<code>`UNIFAI_AGENT_TYPE`</code>	<code>`network`</code>
<code>`UNIFAI_AGENT_ID`</code>	Unique id, e.g. <code>`network-proxy-1`</code>
<code>`UNIFAI_LISTEN_HOST`</code>	<code>`0.0.0.0`</code>
<code>`UNIFAI_PAC_ADVERTISE_ADDR`</code>	<code>`proxy.company.local:8082`</code>
<code>`UNIFAI_FAIL_OPEN`</code>	<code>`0`</code> = fail closed when backend down

5.3 Laptop Guard config (Windows)

Typical fields (JSON / installer config):

Field	Meaning
----- -----	
<code>`backend_url`</code>	UnifAI public URL
<code>`server_mode`</code>	<code>`false`</code> for laptop
<code>`agent_type`</code>	<code>`endpoint`</code>
<code>`listen_host`</code>	<code>`127.0.0.1`</code>
<code>`proxy_addr`</code>	local port Local MITM listen
Fleet pull	From <code>`browser_guard_fleet_config`</code> via heartbeat

5.4 MCP library config

Setting	Value
---------	-------

----- -----
UI → MCP Library → Settings → URL `file:///app/configs/mcp-library.json`
Or empty Uses server default (same file URL)
Sync Force Sync after deploy

5.5 SMTP / Auth (login emails)

Configure SMTP in UnifAI Settings / env as used by your deployment so:

- Registration accept/reject emails
- New-device login notices

(Exact SMTP keys follow your `config.json` / settings UI — verify after deploy.)

5.6 Observability connectors (high level)

Each connector needs **its own** destination credentials in Observability UI:

Connector Typical config
----- -----
Maxim API keys / request header capture
Datadog API key, site, agent addresses
BigQuery project, dataset, table, SA key
Kafka brokers, topic, SASL/TLS
Pub/Sub project, topic, SA key
New Relic license/API keys
OTEL collector URL + Headers (auth to collector)

6. Verify matrix

Layer Check Pass look like
----- ----- -----
Server `/health` `{ "status": "ok" }`
Server Browser AI API JSON targets/rules
Windows Guard `127.0.0.1:18085` Correct version
Windows Guard Agents UI Heartbeat + version
Windows Guard Target site Log in Prompt Logs
Network proxy PAC URL PAC body, not HTML
Mac — Pending agent; or corp PAC workaround
MCP Library ~122 + logos
Headers MCP Install with API key Connected (valid key)

7. File map (quick)

```
.env / .env.example
docker-compose.yml
configs/config.json
```

```
configs/mcp-library.json
apps/browser-guard/README.md
apps/browser-guard/HYBRID_DEPLOY.txt
apps/browser-guard/release/INSTALL_WINDOWS.txt
apps/browser-guard/installer/IT_README.txt
apps/browser-guard/installer/EMPLOYEE_README.txt
deploy/helm/unifai/README.md
docs/UNIFAI_STATUS_ROADMAP.md          # feature roadmap
docs/UNIFAI_IMPLEMENTATION_CONFIG.md   # this file
```

8. Immediate rollout order

1. **Server:** set ``.env`` → compose up → health + Browser AI JSON APIs.
2. **Windows:** ship ``UnifAI_Guard_Setup.exe`` / EXE → Agents heartbeat.
3. **Network (optional):** start ``unifai_broswer_proxy`` + PAC + CA.
4. **Mac:** use network PAC workaround; schedule Mac agent build (Section 4).
5. **Config polish:** MCP file URL, SMTP, observability credentials.

End of Implementation & Configuration Guide.